

More Data Manipulation

pd.read_csv()

- The `pd.read_csv()` function is a core function in pandas for reading data from a CSV file into a DataFrame. It allows us to load CSV data and provides multiple options to customise the way the data is read.

Basix syntax:

```
df = pd.read_csv('filename.csv')
```

When working with datasets that contain dates, it is important to make sure that the date columns are interpreted as datetime objects rather than plain strings. This allows you to perform operations such as filtering by date, extracting specific time periods (like months or years), or plotting time-series data.

The `parse_dates` argument tells pandas to recognise and convert the specified columns into datetime objects automatically. This simplifies date manipulation later in your analysis.

Example syntax:

```
df = pd.read_csv('shopping.csv', parse_dates=['Date'])
```

Handling Date Formats with dayfirst

- By default, pandas follows the convention of "month/day/year" (**MM/DD/YYYY**) for date parsing, which is common in the United States. However, in many other parts of the world, the date format is "day/month/year" (**DD/MM/YYYY**).
- The `dayfirst=True` argument specifies that the first part of the date represents the day, not the month, which is useful when your dataset follows this format.
- Example:
 - `df = pd.read_csv('shopping.csv', parse_dates=['Date'], dayfirst=True)`
 - For example, if your data has dates like "15/01/2023", `dayfirst=True` will interpret it as January 15, 2023, rather than the 1st of May 2023

Keeping the Original Date Column for Future Operations

```
df['Date_original'] = df['Date']
```

- This line creates a backup of the original Date values. The Date column from the DataFrame df is being copied into a new column called Date_original. The dayfirst=True argument specifies that the first part of the date represents the day, not the month, which is useful when your dataset follows this format.

Converting the Date to a Monthly Period

```
df['Month'] = df['Date_original'].dt.to_period('M')
```

- Here, a new column Month is created by converting the Date_original column into a period object representing months.
- .dt.to_period('M') is used to convert the datetime values into monthly periods.
- Example:
 - if Date_original is 2023-01-15, the resulting value in Month will be 2023-01 (representing the entire month of January 2023, not just a specific day).
- **Grouping by Time:** When you want to analyse data over time, it is common to group by larger time periods such as months, quarters, or years. By converting dates into periods, it becomes easy to group the data by month and calculate things like monthly totals, monthly averages, or other monthly statistics.
- **Visualisation:** Plotting data by months (or any other period) is a common use case for visualizing trends over time, such as sales trends, website traffic, or other metrics that naturally fluctuate over longer intervals.

Handling Missing Values in Data

- In pandas, missing values are typically represented as **NaN** (Not a Number).
- **df.isnull().sum()**: This method shows how many missing values exist in each column.
- **df.info()**: This provides a summary of the DataFrame, including counts of non-null entries.
- If a column or row contains too many missing values, or if the missing data is not essential for your analysis, you can drop them.
 - Drop rows: **df.dropna(inplace=True)**
 - Drop columns: **df.dropna(axis=1, inplace=True)**

Handling Missing Values in Data

- You can also handle missing values by:
- **Filling with a Specific Value:** You can fill missing values in specific columns with a default or placeholder value:
 - `df.fillna({'Column1': 0, 'Column2': 'Unknown'}, inplace=True)`

In this example, numerical columns are filled with 0, and categorical columns (like a product name) are filled with 'Unknown'

- **Filling with Mean/Median/Mode:** For numerical data, a common technique is to fill missing values with the mean, median, or mode (most frequent value) of the column:
 - `df['Price'].fillna(df['Price'].mean(), inplace=True)`
 - `df['Quantity'].fillna(df['Quantity'].median(), inplace=True)`

Handling Missing Values in Data

- In this example, the missing values in the '**Product**' column are filled with '**Unknown**', and the missing numerical values in '**Quantity**', '**Price**', and '**Total**' are filled with **0** or **0.0**.
- This ensures that the dataset is complete and ready for further analysis, such as calculating totals or generating visualisations.

```
# Filling missing values in specific columns
df.fillna({
    'Product': 'Unknown',
    'Quantity': 0,
    'Price': 0.0,
    'Total': 0.0
}, inplace=True)

# Printing the cleaned DataFrame
print(df.head())
```

More data manipulation – date format and missing data

1. Load the dataset from a file called "**shopping.csv**".
2. Clean the data by handling missing values.
3. Calculate a new column (Total) based on the price and quantity.
4. Create a visualisation that shows the total sales per product using a bar chart.
5. Plot the total sales over time using a line chart, grouping by the Month.